"FLOCK SIMULATION": Relazione sul progetto d'esame per il corso di "Programmazione per la fisica"

Maltoni Marco, Pigoni Samuele, Santolini Sofia, Zani Luca

07/07/2023

1 Scelte progettuali ed implementative

Il progetto ha lo scopo di rappresentare una simulazione del comportamento di stormi di uccelli in uno spazio bidimensionale. Il modello consiste in oggetti chiamati "boid", che grazie all'applicazione di tre regole che modificano la loro velocità, tendono a convergere ed allinearsi con i boid vicini ad essi, pur non sovrapponendosi, creando così un insieme caratterizzato da un movimento ordinato e coeso. Per implementare ciò in linguaggio C++, sono state create tre differenti classi, con scopi differenti:

- una classe "Vector2", che corrisponde ad una coppia di valori di tipo "float"(scelta derivata dal fatto che la libreria grafica utilizzata per rappresentare il modello, "SFML 2.5.1", lavora con questo tipo primitivo), pensata per facilitare il calcolo vettoriale tra posizioni e velocità dei boid;

- una classe "Boid", che nella parte privata contiene: due elementi di tipo "Vector2", che rappresentano posizione e velocità dei boid; due "float", il primo che rappresenta la velocità massima che essi possono raggiungere, così da evitare comportamenti anomali, ed il secondo che corrisponde a metà dell'angolo di vista all'interno del quale sono individuati i boid "vicini"; un oggetto di tipo "sf::CircleShape", tratto dalla libreria grafica ed opportunamente modificato con i metodi della stessa, così da fare in modo che i boid siano rappresentati da forme triangolari. All'interno della parte pubblica si trovano invece sia la definizione delle tre regole di volo, sia metodi per ruotare e spostare i singoli oggetti di tipo Boid durante la rappresentazione grafica. Le regole prendono come argomenti dei parametri, che sono contenuti nella parte privata della classe definita successivamente ed inseriti come input nel "main".

- una classe "Flock", contenente nella parte privata i parametri che le tre regole prendono come argomenti ed un vettore di oggetti di tipo "Boid", di modo che si possa evolvere nel tempo l'insieme di tutti gli elementi contenuti in esso. I metodi della classe prendono in input un generico intervallo di tempo, e fanno in modo che velocità e posizione di ogni boid vari nel tempo tramite l'applicazione delle tre regole ad ognuno di essi, ed inoltre curano anche l'evoluzione grafica del modello ed il calcolo di alcuni parametri rappresentativi dello stormo.

Un ultimo file, "main.cpp", oltre a contenere i messaggi di input ed output, ha al suo interno l'implementazione delle parti riguardanti la visualizzazione del modello tramite la libreria grafica: esso contiene la definizione di una finestra grafica, di tipo "sf::RenderWindow", la gestione degli eventi e del tempo ed infine la parte funzionale al riempimento ed alla cancellazione della finestra ad ogni frame.

Scendendo più nel dettaglio:

- la classe "Vector2" contiene metodi interni che hanno lo scopo di rappresentare tutte le operazioni necessarie tra vettori bidimensionali, oltre alle relazioni di uguaglianza e disuguaglianza.

- la classe "Boid", invece, ha al suo interno tre metodi che rappresentano le tre regole di volo fornite dal modello, una per la "separazione", una per l'"allineamento" ed una per la "coesione"; inoltre sono stati implementati metodi per determinare l'angolo di inclinazione a partire dalle componenti della velocità, per limitare la stessa velocità se supera il valore massimo, per cambiare i parametri privati degli oggetti di tipo Boid con metodi esterni alla classe (ad esempio è stato necessario all'interno di "Flock") e per settare tutte le caratteristiche grafiche di "boidshape", la forma che rappresenta graficamente i boid, oltre che per muoverla e ruotarla. Sono inoltre presenti un metodo per calcolare l'angolo tra la posizioni di due boid ed un metodo per stabilire se due boid sono "vicini": per essere definiti tali la distanza tra essi deve essere inferiore ad un certo parametro ed uno di essi deve trovarsi inoltre all'interno dell'angolo di vista dell'altro.

- in "Flock", invece, oltre a metodi che permottono di applicare alcune funzionalità dei vettori contenenti tipi primitivi al vettore della classe, che ha al suo interno oggetti di tipo "user-defined", sono stati implementati metodi per calcolare quanti boid sono "vicini" ad uno dato (ciò significa che la distanza tra le loro posizioni è minore di una dei parametri della classe, "d", trattato più approfonditamente nei paragrafi successivi, ed uno di essi si deve trovare all'interno dell'angolo di vista dell'altro), per

calcolare la posizione del centro di massa ed infine per evolvere nel tempo gli elementi del vettore applicando le tre regole ad un dato boid (con dei "for loop" poi queste funzionalità possono essere facilmente estese a tutti gli elementi presenti nel vettore). Infine è presente un metodo che contiene tutte le azioni che devono essere compiute ad ogni frame: in questo modo è possibile evolvere il sistema di un generico intervallo di tempo ottenendo per ogni boid una nuova velocità e posizione, oltre allo spostamento compiuto. Inoltre essi sono anche mantenuti all'interno dei bordi ed al di sotto di una velocità limite.

- nel "main" infine sono gestiti gli eventi "spinta del tasto destro/sinisto del mouse", che permettono di inserire nuovi elementi durante la simulazione, sia all'interno della finestra che in un oggetto di classe "Flock". Inoltre viene impostato lo sfondo della finestra ed infine sono mossi e disegnati i boid presenti all'interno del sistema.

  Il tempo è gestito tramite un oggetto "sf::Clock", definendo un oggetto "sf::Time" con il metodo ".restart()", che azzera ad ogni frame lo scorrere del tempo, e dato in input ai metodi che prendono in input oggetti di tipo "float" della classe "Flock", tramite il metodo ".asSeconds()", che ritorna il tempo trascorso tra l'inizio e la fine di un frame in secondi.
  Il sistema fornisce anche la possibilità di inserire dei predatori, a cui è applicata la sola regola di coesione, però escludendoli dal calcolo del centro di massa, così che loro convergano verso i boid vicini ma che il contrario non avvenga, mentre i boid tendono a fuggire da essi grazie alla regola di separazione. Per distinguere boid e predatore è fondamentale il metodo ".getFillColor()" della libreria grafica: i boid sono di colore rosso, i predatori di colore nero, e nei metodi questa distinzione è posta come condizione per l'inclusione o esclusione di un oggetto dallo svolgimento di una determinata azione.
  Infine, il comportamento ai bordi è stato gestito in due maniere differenti, necessità dovuta a problemi incontrati durante l'implementazione del modello: infatti, dopo l'introduzione dei predatori, ai limiti della finestra la tendenza dei boid a fuggire dal predatore era in contrasto con la prima scelta implementativa compiuta, cioè di fare in modo che i boid rimanessero all'interno dei bordi sommando ad essi una velocità opposta in segno quando toccavano uno di essi. I boid ai bordi della finestra possono quindi assumere due tipi di comportamento:

  - quando un gruppo di boid vicini non contiene un predatore, essi rimangono all'interno dei bordi, come se "rimbalzassero" su essi;

  - quando un gruppo di boid contiene uno o più predatori, i boid vengono spostati al lato opposto della finestra mentre i predatori vengono

respinti dalle pareti. Ciò permette alla simulazione di procedere più fluida, evitando che i boid respinti dalla parete inglobino il predatore creando una situazione di stallo.

2 Istruzioni su come compilare, testare ed eseguire

Inizialmente, è necessario scaricare i file inerenti alla libreria grafica utilizzata per rappresentare la simulazione, "SFML 2.5.1", attraverso il seguente comando: "sudo apt-get install libsfml-dev".
Successivamente si può proseguire con l'esecuzione del programma o dei test. Il programma contiene un file "CMake", che deve essere opportunamente modificato a seconda dell'azione che si vuole compiere:

- per eseguire il programma, si deve entrare nel file "CMakeLists.txt" e commentare con un cancelletto le righe 32, 33 e 36, mentre le righe 1, 2, 5, 7, 10, 11, 12, 15, 18, 19, 21, 23, 25, 29 e 38 devono essere lasciate prive di cancelletto (le rimanenti righe, contenenti stringhe di testo, devono rimanere sempre commentate). Successivamente si deve aprire il terminale e digitare i seguenti comandi: "cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug", "cmake –build build", "./build/flock". Perchè il programma venga eseguito correttamente, oltre a tutti i file è necessario che nella directory sia presente anche l'immagine di sfondo, denominata "cielo.jpg".

- per testare, invece, rispetto alla situazione precedente devono essere commentate le righe 23, 25, mentre le righe 32, 33 e 36 devono essere lasciate prive di cancelletto. Inoltre, la riga 32 va ulteriormente modificata a seconda di quale file di test si voglia svolgere: per "vector2_test.cpp" scrivere all'interno delle parentesi tonde, di seguito a "flock.t", "vector2.cpp vector2_test.cpp", mentre per "boid_test.cpp" digitare "vector2.cpp boid.cpp boid_test.cpp", ed infine per "flock _test.cpp" inserire "vector2.cpp boid.cpp flock.cpp flock_test.cpp". Alla fine dei precedenti passaggi, è necessario aprire il terminale e digitare i seguenti comandi: "cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug", "cmake –build build", "./build/flock.t";

- se si vuole infine disegnere i grafici "ROOT" sull'andamento delle velocità e delle distanze medie tra i boid, aprire "ROOT" da terminale con il comando "root" e digitare ".x graph.C": la macro "graph.C" sarà eseguita.

3 Input ed output

- Input: il programma richiede in input i valori dei cinque parametri (paramentri ottimali, corrispondenti ai valori tenuti costanti durante l'implementazione del programma, sono 100, 30, 0.05, 0.5, 0.0005), che sono:

  - "d", che rappresenta la massima distanza alla quale due boid vengono considerati "vicini", quindi la distanza sotto alla quale si applicano le tre regole. E' permesso digitare un valore all'interno del seguente range: [50; 200];

  - "ds", che corrisponde alla massima distanza a cui si applica la regola di "separazione". E' permesso digitare un valore all'interno del seguente range: [25; 40];

  - "s", che rappresenta il fattore moltiplicativo della legge di "separazione". E' permesso digitare un valore all'interno del seguente range: [0.005; 0.08];

  - "a", che rappresenta il fattore moltiplicativo della regola di "alline amento". E' permesso digitare un valore all'interno del seguente range: [0.2; 0.8];

  - "c", che rappresenta il fattore moltiplicativo della legge di coesione. E' permesso digitare un valore all'interno del seguente range: [0.0001; 0.001]

  Una volta aperta la finestra sarà necessario premere il tasto sinistro del mouse o touchpad per inserire all'interno della simulazione un boid, mentre con il tasto destro si potrà fare apparire un predatore. Seppur nessuno dei due oggetti abbia un limite massimo di quantità, per non rendere troppo caotica la simulazione si consiglia di non aggiungere un numero troppo elevato di predatori.

- Output: grazie al metodo "state()" della classe "Flock" il programma restituisce in output i valori della media dei moduli delle velocità dei boid e della distanza media tra essi (calcolata senza ripetizioni, cioè consiederando la distanza tra due boid una sola volta), seguite dalle rispettive deviazioni standard sulla media, e questi valori vengono stampati sul terminale ad ogni secondo appena il numero di boid è sufficiente. Un esempio di output sul terminale è: "Medium velocity: 0.448897 +/- 0.0590251; Medium distance among boids: 556.846 +/- 492.186;"

Una volta chiusa la finestra grafica, azione possibile grazie al tasto con raffigurata una "x", appariranno sul terminale dei messaggi di errore dovuti a "memory leak": essi sono causati dalla libreria grafica di "SFML" ma non rappresentano un problema per il funzionamento del programma.

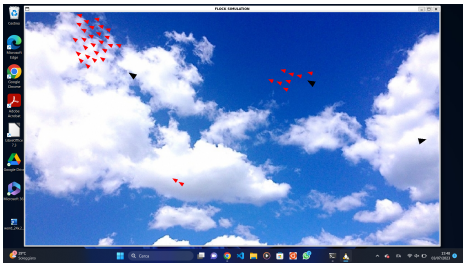

Figure 1: Esempio di come appare la finestra grafica durante la simulazione: i boid sono di colore rosso, mentre i predatori sono identificati col nero.

4 Interpretazione dei risultati ottenuti

Inserendo dei valori per i parametri all'interno dei range indicati, si ottiene un movimento coeso dei boid, che tendono ad avvicinarsi ed allinearsi. Inserendo un predatore, esso tende ad inseguirli, ma senza mai sovrapporsi ad essi, anche grazie al fatto che essi raggiungendo i bordi vengono spostati dalla parte opposta della finestra. La velocità dei boid non raggiunge valori troppo elevati grazie al metodo "limit_velocity()" della classe "Boid", così da rendere la simulazione ordinata e non eccessivamente caotica. Inoltre, attraverso il programma di analisi dati "ROOT", scrivendo la macro "graph.C" ed inserendo due insiemi di dati ottenuti dall'output del programma riguardante una casuale simulazione nei file "dati.txt" e "dati2.txt", sono stati ottenuti i seguenti due grafici:

- dal primo grafico si evince come inserendo dei nuovi boid all'interno del sistema, la velocità media aumenti, per poi tendere ad un valore costante all'avvicinarsi dei boid e man mano che la creazione dello stormo procede;

- dal secondo grafico invece si nota come la distanza media dei boid continui a scendere fino a stabilizzarsi quando essi hanno raggiunto il minimo consentito dalla regola di separazione. Entrambi questi comportamenti sono attribuibili ad uno stormo.

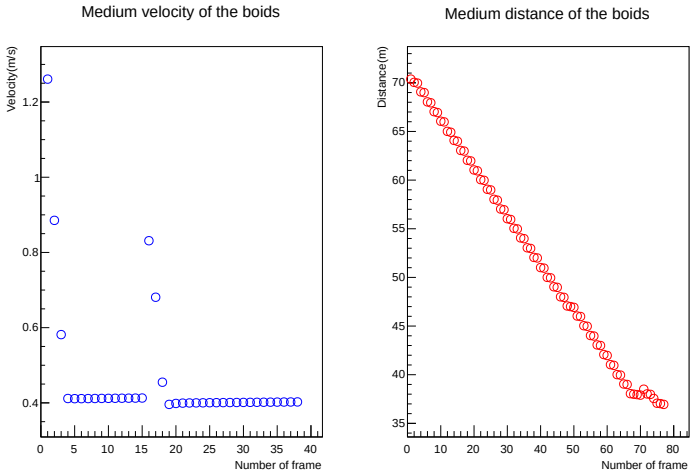

Figure 2: Grafici disegnati con ROOT: sulle ascisse si trova il numero di frame, mentre sulle ordinate appaiono, in ordine: la media dei moduli delle velocità, la distanza media dei boid, tratti da una casuale simulazione.

5 Strategie di testing

Per testare le funzioni e i metodi definiti all'interno del programma sono state utilizzate due modalità:

- utilizzo di "DOCTEST", incluso nel programma tramite il file "docte st.h", con cui è stato possibile creare situazioni ad hoc per controllare che i risultati attesi dai vari componenti del programma fossero effettivamente restituiti come output. Questa parte di "testing" corrisponde ai file il cui nome termina con "_test.cpp", ed è eseguibile attraverso la seconda strategia di compilazione descritta nel "paragrafo 2".

- alcuni metodi, tuttavia, non si prestavano all'uso di "DOCTEST": è il caso soprattutto dei metodi riguardanti l'estetica delle forme dei boid ed il loro movimento o rotazione. La correttezza di queste funzioni, a cui si aggiunge anche il metodo "in_limits()", è stata verificata solo nel momento in cui si è iniziato ad eseguire il programma, controllando che i boid avessero le caratteristiche richieste dall'implementazione, cioè che si muovessero e ruotassero correttamente (ad esempio allineandosi sempre alla direzione del loro moto) e che stessero all'interno dei bordi, tranne quando fossero inseguiti

da un predatore. La correttezza della rappresentazione grafica della simulazione dopo svariate prove è stata considerata una prova soddisfacente della correttezza delle funzioni sopra citate.

6 Correzioni e miglioramenti per l'appello di settembre

- Utilizzo di "Git" e "Github": per tenere traccia dei miglioramenti apportati, raggrupparli e riuscire a tornare alla versione precedente del programma dopo un cambiamento errato in modo facile e veloce sono stati utilizzati gli strumenti "Git" e "Github". Attraverso l'utilizzo di "Git" è stato possibile apportare cambiamenti al programma in modo più coerente e sicuro, e creando una repository remota su "Github" è stata migliorata la visibilità del programma, aprendo la strada a una possibile collaborazione per il suo miglioramento. L'indirizzo della repository pubblica su "Github" è: "https://github.com/Marcomaltoni/Boid_simulation".

- Cambiamenti suggeriti al primo colloquio orale:

  - Miglioramento del metodo "state()": la stampa dei valori medi è stata permessa solo con più di due boid sullo schermo, così da evitare che apparissero sul terminale svariate righe in cui i valori medi erano nulli. Inoltre è stato cambiato il tipo di ritorno della funzione, passando da un "array" di oggetti di tipo "Vector2" ad una "struct", così che la sintassi con cui erano rappresentati i valori di ritorno fosse più chiara e significativa.

  - Implementazione di una "class invariant" per la classe "Flock": i parametri presi in input da un oggetto di tipo "Flock" dovevano appartenere ad un range ben definito perchè il comportamento dei boid fosse coerente con quello di uno stormo. Per questo motivo è stato impedito che ci fosse la possibilità di inserire valori diversi da quelli permessi attraverso l'uso di "assert" all'interno del costruttore della classe "Flock".

  - Implementazione degli operatori uno in funzione dell'altro: per ridurre al minimo gli ipotetici cambiamenti da apportare in caso di modifica di un metodo della classe, gli operatori sono stati definiti a partire da quelli già definiti in precedenza: ad esempio l'"operatore+" della classe "Vector2" è stato definito utilizzando la logica dell'"operatore+=" definito prima di esso.

- Correzione dei test: in alcuni test è stata aggiunta la funzionalità "doctest::Approx" poichè essi erano relativi a funzioni che tornavano valori "floating point". Ciò era necessario per specificare l'approssimazione a cui erano soggetti i risultati.

- Miglioramenti ad alcuni metodi: sono state rimosse conversioni senza utilità messe in atto tramite la funzionalità "static_cast<>", nel metodo "is_predator()" è stata eliminata una doppia iterazione attuata con un ciclo for ed è stato preferito un algoritmo.

- Utilizzo di una sintassi più espressiva: i nomi degli oggetti utlizzati e delle funzioni sono stati variati con l'obiettivo di usare termini più coerenti con la logica del programma e significativi. Ad esempio in una funzione che agisce su un boid specifico e prende come argomento un altro boid, invece di chiamare i due oggetti "x" e "b", si è optato per "chosen_boid" e "other_boid".

- Utilizzo maggiore di "assert": sono stati aggiunti vari "assert" all'interno del programma per assicurarsi che la comprensione di esso posseduta dallo sviluppatore fosse coerente con i risultati ottenuti. Al contrario, se le espressioni all'interno di essi non fossero state soddisfatte il programma sarebbe terminato.

- Ulteriori correzioni al metodo "state()": la funzione è stata ulteriormente corretta, così che i valori tornati fossero le incertezze sulle medie (si è diviso per la radice del numero di boid presenti all'interno della simulazione) e che essi fossero stampati a schermo a intervalli di un secondo, e non più ad ogni frame.

- Miglioramenti alla regola di separazione: la regola di separazione è stata migliorata attraverso il metodo "close_boids()", che restituiva risultati errati quando il boid considerato era un predatore. In questo modo i boid inseguiti non tendono più a fuggire verso l'angolo in alto a sinistra, ma fuggono dalla parte opposta alla direzione di arrivo del predatore.

- Implementazione di un "angolo di vista per i boid": il modo in cui i boid individuavano i vicini è stato variato, aggiungendo alla condizione sulla distanza la richiesta che il boid fosse all'interno di un settore circolare con angolo di ampiezza uguale all'angolo giro meno il doppio del supplementare dell'angolo di vista fornito come parametro all'oggetto di tipo "Boid", e di raggio il parametro che rappresenta la distanza di vicinanza. Questa condizione è stata richiesta per le regole di "allineamento" e "coesione", mentre per la regola di "sepa-

razione" è stata scelta la precedente implementazione, per facilitare l'individuazione dei predatori da parte dei boid.